Workflow Summary

1. **Data Preparation**: Load and preprocess datasets.
2. **Collaborative Filtering**: Build recommendations using SVD.
3. **Dynamic User Input**: Accept new ratings and generate personalized recommendations.
4. **Content-Based Filtering**: Address cold-start problems using genre similarity.

```
# Import Libraries
import pandas as pd
import numpy as np
from sklearn.decomposition import TruncatedSVD
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Load Datasets
links_df = pd.read_csv('links.csv')
movies_df = pd.read_csv('movies.csv')
ratings_df = pd.read_csv('ratings.csv')
tags_df = pd.read_csv('tags.csv')
print(links_df.head())
print(movies_df.head())
print(ratings_df.head())
print(tags_df.head())
```

```
   movieId  imdbId   tmdbId
0        1  114709    862.0
1        2  113497   8844.0
2        3  113228  15602.0
3        4  114885  31357.0
4        5  113041  11862.0
   movieId                               title  \
0        1                    Toy Story (1995)
1        2                      Jumanji (1995)
2        3             Grumpier Old Men (1995)
3        4            Waiting to Exhale (1995)
4        5  Father of the Bride Part II (1995)

                                        genres
0  Adventure|Animation|Children|Comedy|Fantasy
1                   Adventure|Children|Fantasy
2                               Comedy|Romance
3                         Comedy|Drama|Romance
4                                       Comedy
   userId  movieId  rating  timestamp
0       1        1     4.0  964982703
1       1        3     4.0  964981247
2       1        6     4.0  964982224
3       1       47     5.0  964983815
4       1       50     5.0  964982931
```

```
   userId  movieId              tag   timestamp
0       2    60756            funny  1445714994
1       2    60756  Highly quotable  1445714996
2       2    60756     will ferrell  1445714992
3       2    89774     Boxing story  1445715207
4       2    89774              MMA  1445715200
```

```
# Prepare User-Item Interaction Matrix
movies_ratings_df = pd.merge(ratings_df, movies_df, on='movieId',
how='inner')
user_item_matrix = movies_ratings_df.pivot_table(index='userId',
columns='movieId', values='rating')
movies_ratings_df
user_item_matrix
```

```
movieId  1       2       3       4       5       6       7       8
\
userId

1           4.0     NaN     4.0     NaN     NaN     4.0     NaN
NaN
2           NaN     NaN     NaN     NaN     NaN     NaN     NaN
NaN
3           NaN     NaN     NaN     NaN     NaN     NaN     NaN
NaN
4           NaN     NaN     NaN     NaN     NaN     NaN     NaN
NaN
5           4.0     NaN     NaN     NaN     NaN     NaN     NaN
NaN
...         ...     ...     ...     ...     ...     ...     ...     ..
.
606         2.5     NaN     NaN     NaN     NaN     NaN     2.5
NaN
607         4.0     NaN     NaN     NaN     NaN     NaN     NaN
NaN
608         2.5     2.0     2.0     NaN     NaN     NaN     NaN
NaN
609         3.0     NaN     NaN     NaN     NaN     NaN     NaN
NaN
610         5.0     NaN     NaN     NaN     NaN     5.0     NaN
NaN

movieId  9       10      ...  193565  193567  193571  193573  193579
193581  \
userId                   ...

1           NaN     NaN  ...     NaN     NaN     NaN     NaN     NaN
NaN
2           NaN     NaN  ...     NaN     NaN     NaN     NaN     NaN
NaN
```

```
3            NaN     NaN  ...     NaN     NaN     NaN     NaN     NaN
NaN
4            NaN     NaN  ...     NaN     NaN     NaN     NaN     NaN
NaN
5            NaN     NaN  ...     NaN     NaN     NaN     NaN     NaN
NaN
...          ...     ...  ...     ...     ...     ...     ...     ...
...
606          NaN     NaN  ...     NaN     NaN     NaN     NaN     NaN
NaN
607          NaN     NaN  ...     NaN     NaN     NaN     NaN     NaN
NaN
608          NaN     4.0  ...     NaN     NaN     NaN     NaN     NaN
NaN
609          NaN     4.0  ...     NaN     NaN     NaN     NaN     NaN
NaN
610          NaN     NaN  ...     NaN     NaN     NaN     NaN     NaN
NaN

movieId  193583  193585  193587  193609
userId
1           NaN     NaN     NaN     NaN
2           NaN     NaN     NaN     NaN
3           NaN     NaN     NaN     NaN
4           NaN     NaN     NaN     NaN
5           NaN     NaN     NaN     NaN
...         ...     ...     ...     ...
606         NaN     NaN     NaN     NaN
607         NaN     NaN     NaN     NaN
608         NaN     NaN     NaN     NaN
609         NaN     NaN     NaN     NaN
610         NaN     NaN     NaN     NaN

[610 rows x 9724 columns]
```

```python
# Collaborative Filtering with SVD
def dynamic_user_recommendation(user_id, new_ratings,
user_item_matrix, movies_df, n_recommendations=5):
    if user_id not in user_item_matrix.index:
        user_item_matrix.loc[user_id] = np.nan
    for movie_id, rating in new_ratings.items():
        user_item_matrix.loc[user_id, movie_id] = rating
    user_item_matrix_filled = user_item_matrix.fillna(0)
    svd = TruncatedSVD(n_components=50, random_state=42)
    svd_matrix = svd.fit_transform(user_item_matrix_filled)
    reconstructed_matrix = np.dot(svd_matrix, svd.components_)
    reconstructed_df = pd.DataFrame(reconstructed_matrix,
index=user_item_matrix.index, columns=user_item_matrix.columns)
    user_predictions = reconstructed_df.loc[user_id]
    unrated_movies = user_item_matrix.loc[user_id]
```

```
[user_item_matrix.loc[user_id].isna()]
    recommendations = user_predictions[unrated_movies.index]
    top_recommendations =
recommendations.sort_values(ascending=False).head(n_recommendations)
    recommended_movies =
movies_df[movies_df['movieId'].isin(top_recommendations.index)]
    return recommended_movies[['movieId', 'title']]

# Example: Dynamic User Input
new_user_id = 1000
new_user_ratings = {1: 5.0, 2: 4.5, 858: 4.0}
personalized_recommendations =
dynamic_user_recommendation(new_user_id, new_user_ratings,
user_item_matrix, movies_df)
personalized_recommendations
```

	movieId	title
224	260	Star Wars: Episode IV - A New Hope (1977)
520	608	Fargo (1996)
615	780	Independence Day (a.k.a. ID4) (1996)
896	1193	One Flew Over the Cuckoo's Nest (1975)
922	1221	Godfather: Part II, The (1974)

```
# Evaluation of Collaborative Filtering Model
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Function to evaluate SVD with RMSE and MAE
def evaluate_svd(user_item_matrix, svd_components=50):
    user_item_matrix_filled = user_item_matrix.fillna(0)
    svd = TruncatedSVD(n_components=svd_components, random_state=42)
    svd_matrix = svd.fit_transform(user_item_matrix_filled)
    reconstructed_matrix = np.dot(svd_matrix, svd.components_)
    reconstructed_df = pd.DataFrame(reconstructed_matrix,
index=user_item_matrix.index, columns=user_item_matrix.columns)

    # Extract true ratings and predicted ratings
    true_ratings = user_item_matrix.values.flatten()
    predicted_ratings = reconstructed_df.values.flatten()
    valid_mask = ~np.isnan(true_ratings)
    true_ratings = true_ratings[valid_mask]
    predicted_ratings = predicted_ratings[valid_mask]

    # Compute RMSE and MAE
    rmse = mean_squared_error(true_ratings, predicted_ratings,
squared=False)
    mae = mean_absolute_error(true_ratings, predicted_ratings)

    return {'RMSE': rmse, 'MAE': mae}

# Evaluate the model
```

```
evaluation_results = evaluate_svd(user_item_matrix)
evaluation_results
```

```
C:\Users\Tech Via\anaconda3\Lib\site-packages\sklearn\metrics\
_regression.py:492: FutureWarning: 'squared' is deprecated in version
1.4 and will be removed in 1.6. To calculate the root mean squared
error, use the function'root_mean_squared_error'.
  warnings.warn(

{'RMSE': 1.998211920371504, 'MAE': 1.538632405144087}
```